



EXPERIMENT -8

Student Name: Vinay Tiwari

Branch: AIT-CSE (AIML)

Semester: 5

Subject Name: Database Management System

UID: 24BAI70919

Section/Group: 24AIT_KRG-1A

Subject Code: 24CSH-298

Q1) A company maintains customer information, product information, and sales order information in three separate tables: customer_master, product_catalog, and sales_orders.

The company wants to provide an order report to an external client. The report must contain the Order ID, Order Date, Customer Name, Product Name, and Final Amount. For every order, calculate:

- Gross Amount = Unit Price × Quantity
- Discount Amount = Gross Amount × Discount Percentage / 100
- Final Amount = Gross Amount – Discount Amount

Create a complex view using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.

The company also wants to provide access to an external client. Create a PostgreSQL user named client_user and give this user only SELECT permission on the view. The client must not have direct access to the three original tables and must not be allowed to modify the view data.

SOFTWARE REQUIREMENTS:

Database Management System:

PostgreSQL Database

Database Administration Tool / Client Tool:

pgAdmin (for PostgreSQL)

EXPERIMENT -8

CODE SCREENSHOTS:

```
CREATE OR REPLACE VIEW vw_ORDER_PROCESSING
AS
SELECT
    O.order_id,
    O.order_date,
    C.full_name,
    P.product_name,

    (P.unit_price * O.quantity)
    -
    (
        (P.unit_price * O.quantity)
        * O.discount_percent / 100
    ) AS final_amount

FROM customer_master C

JOIN sales_orders O
    ON C.customer_id = O.customer_id

JOIN product_catalog P
    ON P.product_id = O.product_id;
```

```
CREATE USER client_user WITH PASSWORD 'client@123';

REVOKE ALL ON customer_master, product_catalog, sales_orders
FROM client_user;

GRANT SELECT ON vw_ORDER_PROCESSING TO client_user;
```

EXPERIMENT -8

OUTPUT SCREENSHOTS:

	order_id integer	order_date date	full_name character varying (50)	product_name character varying (50)	final_amount numeric
1	1	2025-09-01	Amit Sharma	Smartphone X100	47500.000000000000000000
2	2	2025-09-02	Priya Verma	Laptop Pro 15	58500.000000000000000000
3	3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.000000000000000000
4	4	2025-09-04	Neha Singh	Smartwatch Fit	27600.000000000000000000
5	5	2025-09-05	Arjun Mehta	Gaming Console	39600.000000000000000000
6	6	2025-09-06	Priya Verma	Smartphone X100	23750.000000000000000000
7	7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.000000000000000000

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand the concept of **views** as virtual tables in PostgreSQL.
- Create a **complex view** by joining three tables using **JOIN** conditions.
- Calculate the **Final Amount** by applying the discount on the gross amount inside the view.
- Hide sensitive columns by exposing only the **required fields** through the view.
- Create a user and control access using **GRANT** and **REVOKE** permissions.

CONCLUSION:

In conclusion, this task demonstrates how views can combine data from multiple tables into a single report while hiding sensitive information. By granting the external user **SELECT** permission only on the view, the company can share the required data securely without giving access to the original tables.

EXPERIMENT -8

Q2) An e-commerce company wants to regularly analyze the performance of its product categories. The company stores product, category, customer, and order information in separate tables. To generate the category performance report, the database needs to perform multiple JOINS, GROUP BY, aggregate functions, DISTINCT counting, calculated values, and CASE statements.

Since the same report is generated frequently, executing the complete analytical query every time can increase database workload. Create a PostgreSQL Materialized View named mv_category_sales.

Also demonstrate that when new order data is inserted, the Materialized View does not automatically reflect the change and must be refreshed. Finally, use EXPLAIN ANALYZE to compare the performance of the original analytical query with the Materialized View query.

Business Rules:

Requirement	Rule
Total Products Sold	SUM(quantity)
Total Orders	COUNT(DISTINCT order_id)
Unique Customers	COUNT(DISTINCT customer_id)
Total Revenue	SUM(quantity × unit_price)
Average Revenue per Order	Total Revenue / Total Orders
Excellent	Revenue ≥ 10,000
Good	Revenue ≥ 6,000 and < 10,000
Average	Revenue ≥ 3,000 and < 6,000
Low	Revenue < 3,000

SOFTWARE REQUIREMENTS:

Database Management System:

PostgreSQL Database

Database Administration Tool / Client Tool:

pgAdmin (for PostgreSQL)

EXPERIMENT -8

CODE SCREENSHOTS:

```
CREATE MATERIALIZED VIEW mv_category_sales_01
AS

SELECT
    c.category_id,
    c.category_name,

    SUM(od.quantity) AS total_products_sold,

    COUNT(DISTINCT od.order_id) AS total_orders,

    COUNT(DISTINCT od.customer_id) AS unique_customers,

    SUM(od.quantity * od.unit_price) AS total_revenue,

    ROUND(
        SUM(od.quantity * od.unit_price)
        / COUNT(DISTINCT od.order_id),
        2
    ) AS avg_revenue_per_order,

    CASE
        WHEN SUM(od.quantity * od.unit_price) >= 10000
        THEN 'Excellent'

        WHEN SUM(od.quantity * od.unit_price) >= 6000
        THEN 'Good'
```

EXPERIMENT -8

```

        THEN 'Good'

    WHEN SUM(od.quantity * od.unit_price) >= 3000
        THEN 'Average'

    ELSE 'Low'
END AS category_performance

FROM category_master c

JOIN product_master p
    ON c.category_id = p.category_id

JOIN order_details od
    ON p.product_id = od.product_id

GROUP BY
    c.category_id,
    c.category_name;

```

```
REFRESH MATERIALIZED VIEW mv_category_sales_01;
```

```
REFRESH MATERIALIZED VIEW
```

OUTPUT SCREENSHOTS:

	category_id character varying (5)	category_name character varying (50)	total_products_sold bigint	total_orders bigint	unique_customers bigint	total_revenue numeric	avg_revenue_per_order numeric	category_performance text
1	C1	Electronics	5	4	3	13000.00	3250.00	Excellent
2	C2	Clothing	4	3	2	10000.00	3333.33	Excellent
3	C3	Books	3	1	1	1500.00	1500.00	Low

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand the difference between a **normal view** and a **materialized view**.
- Create a materialized view using **multiple JOINS**, **GROUP BY** and **aggregate functions**.

EXPERIMENT -8

- Use **COUNT(DISTINCT)** and **ROUND()** to calculate orders, customers and average revenue.
- Classify category performance using a **CASE** statement based on total revenue.
- Update stored results of a materialized view using **REFRESH MATERIALIZED VIEW**.

CONCLUSION:

In conclusion, this task demonstrates how a materialized view stores the result of a complex analytical query physically, so the report can be read quickly without re-running all the joins and aggregations each time. Since the stored data does not update automatically, the view must be refreshed after new orders are inserted to keep the report accurate.